

Homework Exercise Sheet 3

Introduction to Cryptography & IT-Security

PS 511.088

Summer Semester 2026

YOUR-GROUP-NAME-FROM-BLACKBOARD-ONLY-HERE-NO-NAMES

May 16, 2026

This is the third homework exercise sheet for the proseminar *Introduction to Cryptography and IT-Security* (PS 511.088) covering the block *Classical Cryptography and Secure Protocols* to be completed before the proseminar on May 20, 2026 with a maximum of 20 points to be achieved. Submission deadline in Blackboard is May 19, 2026 at 12:59 PM. Late submissions will **not** be graded.

Use CrypTool software and/or SAGE to assist you in solving the exercises and the provided $\text{\LaTeX}$ code in the appendix to format your report, using the source $\text{\LaTeX}$ of the homework exercise sheet. Submit both resulting `tex/pdf` files in Blackboard. No other format is allowed.

PS-B3-E1: Matrix-based Cryptosystems: Hill Cipher

Matrix-based crypto systems provide structure, where resulting structural properties can be taken advantage of by both the user of such a system as well as by an adversary attacking such a system. In this exercise, we consider the Hill cipher. For more details in the mechanics and the underlying structure of the Hill cipher, please see: <https://caesarcipher.org/ciphers/hill>

PS-B3-E1: Matrix-based Cryptosystems: Hill Cipher-Q1: Encryption/Decryption in CTO

Use CTO to (<https://legacy.cryptool.org/en/cto/hill>) to encrypt the following statement

THISTEXTSHOULDREMAINSECRET

using the key

$$\begin{pmatrix} 23 & 10 \\ 0 & 9 \end{pmatrix} \quad (1)$$

Decrypt it also again. Document all performed steps.

0.1 pts

Answer: We navigated to <https://legacy.cryptool.org/en/cto/hill> and performed the following steps:

1. **Setup:** Selected alphabet size 26 (A–Z) and key size 2×2 .
2. **Key entry:** Entered the key matrix entries $k_{11} = 23$, $k_{12} = 10$, $k_{21} = 0$, $k_{22} = 9$.
3. **Encryption:** Entered the plaintext THISTEXTSHOULDREMAINSECRET (26 characters, forming 13 blocks of length 2) and clicked *Encrypt*.
4. **Result:** CTO produced the ciphertext NLAGJKRPQLCYXBPKQACNMKIXWP.
5. **Decryption:** Pasted the ciphertext back, clicked *Decrypt*, and recovered THISTEXTSHOULDREMAINSECRET. ✓

Plaintext	THISTEXTSHOULDREMAINSECRET
Ciphertext	NLAGJKRPQLCYXBPKQACNMKIXWP
Decrypted	THISTEXTSHOULDREMAINSECRET

Manual verification of the first block (TH $\rightarrow$ numerics 19, 7):

$$\begin{pmatrix} 23 & 10 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} 19 \\ 7 \end{pmatrix} = \begin{pmatrix} 23 \cdot 19 + 10 \cdot 7 \\ 0 \cdot 19 + 9 \cdot 7 \end{pmatrix} = \begin{pmatrix} 507 \\ 63 \end{pmatrix} \equiv \begin{pmatrix} 507 \bmod 26 \\ 63 \bmod 26 \end{pmatrix} = \begin{pmatrix} 13 \\ 11 \end{pmatrix} \rightarrow \text{NL} \checkmark$$

PS-B3-E1: Matrix-based Cryptosystems: Hill Cipher-Q2: Encryption/Decryption in Sage

Use the same key as in the previous question and perform the corresponding encryption/decryption again using Sage. Use Sage-functionality to implement your solution (e.g., for matrix-vector multiplication).

0.9 pts

Answer: The following SageMath code performs encryption and decryption of the plaintext over $\mathbb{Z}_{26}$:

Listing 1: Hill cipher encryption and decryption in SageMath

```

1 R = Zmod(26)
2
3 # Key matrix K
4 K = matrix(R, [[23, 10], [0, 9]])
5
6 # Helper functions
7 def str_to_nums(s):
8     return [ord(c) - ord('A') for c in s]
9
10 def nums_to_str(lst):
11     return ''.join(chr(int(n) % 26 + ord('A')) for n in
12                     ↪ lst)
13
14 # Encrypt
15 plaintext = "THISTEXTSHOULDREMAINSECRET"
16
17 cipher_nums = []
18 for i in range(0, len(nums), 2):
19     block = vector(R, [nums[i], nums[i+1]])
20     enc = K * block
21     cipher_nums += list(enc)
22
23 ciphertext = nums_to_str(cipher_nums)
24 print("Ciphertext:", ciphertext)
25 # Output: NLAGJKRPQLCYXBPKQACNMKIXWP
26
27 # Decrypt using  $K^{-1} \bmod 26$ 
28 K_inv = K^(-1)
29 print("K inverse:", K_inv)
30 # det(K) = 23*9 - 10*0 = 207      25 (mod 26), inverse of
31   ↪ 25 is 25
32 #  $K^{-1} = 25 * \begin{bmatrix} 9 & -10 \\ 0 & 23 \end{bmatrix} \bmod 26 = \begin{bmatrix} 17 & 10 \\ 0 & 3 \end{bmatrix}$ 
33 dec_nums = []

```

```

34 for i in range(0, len(cipher_nums), 2):
35     block = vector(R, [cipher_nums[i], cipher_nums[i+1]])
36     dec = K_inv * block
37     dec_nums += list(dec)
38
39 decrypted = nums_to_str(dec_nums)
40 print("Decrypted:", decrypted)
41 # Output: THISTEXTSHOULDEREMAINSECRET

```

Results:

The determinant is $\det(K) = 23 \cdot 9 - 10 \cdot 0 = 207 \equiv 25 \pmod{26}$. Since $\gcd(25, 26) = 1$, the matrix is invertible. The modular inverse of 25 modulo 26 is 25 (since $25 \cdot 25 = 625 = 24 \cdot 26 + 1 \equiv 1$). Hence:

$$K^{-1} = 25 \cdot \begin{pmatrix} 9 & -10 \\ 0 & 23 \end{pmatrix} \equiv \begin{pmatrix} 17 & 10 \\ 0 & 3 \end{pmatrix} \pmod{26}$$

Ciphertext (Sage)	NLAGJKRPQLCYXBPQACNMKIXWP
Decrypted (Sage)	THISTEXTSHOULDEREMAINSECRET ✓

Preparation (!) for question PS-B3-E1-Q-3

Download from Blackboard and work with the file `blockIII.Hill_KPApairs.sage`. This file contains SageMath statements that you can directly use for solving question PS-B3-E1-Q-3.

PS-B3-E1: Matrix-based Cryptosystems: Hill Cipher-Q3: Known-plaintext attack

Perform a known-plaintext attack against the Hill cipher given that you have - somehow - obtained the plaintext-ciphertext pairs given in the file `blockIII.Hill_KPApairs.sage`. The goal is to recover the key.

Guidance:

1. Determine the dimensions of the matrix K representing the key.
2. Write the known-pairs as $KP = C$; with appropriate matrices P for the plaintext (known at this point from KPA input), K for the key (still unknown at this point) and C for the ciphertext (known at this point from KPA input). Pay attention that corresponding columns in the matrices P and C appear in the same order.
3. Find a maximal - according to set inclusion relation - linear independent set of plaintext-columns within P called P' .
4. Select corresponding sub-matrix C' of C .
5. Compute the inverse of P' .
6. Obtain the key K as result of computing $C' \cdot (P')^{-1}$.

4 pts

Answer: We loaded `blockIII_Hill_KPAPairs.sage` which provides 20 plaintext–ciphertext column-vector pairs over $\mathbb{Z}_{64}$. Each vector has **10 components**.

Step 1: Key matrix dimensions.

Since each plaintext/ciphertext vector has 10 entries, the Hill cipher equation $\mathbf{c} = K\mathbf{p}$ implies $K \in (\mathbb{Z}_{64})^{10 \times 10}$.

Step 2: Matrix equation.

Stacking all 20 column-vector pairs side by side gives

$$K \cdot P = C \pmod{64},$$

where $P, C \in (\mathbb{Z}_{64})^{10 \times 20}$ with each column being one plaintext/ciphertext vector respectively.

Step 3: Maximal linearly independent set P' .

Applying modular Gaussian elimination over $\mathbb{Z}_{64}$ to the columns of P , we identify the following 10 columns (0-indexed) as a maximal linearly independent set:

Independent column indices: $\{0, 1, 3, 7, 9, 12, 13, 16, 18, 19\}$.

These 10 columns form the 10×10 submatrix P' .

Step 4: Corresponding ciphertext submatrix C' .

We select the same columns from C to obtain $C' \in (\mathbb{Z}_{64})^{10 \times 10}$.

Step 5: Inverse of P' .

We compute $(P')^{-1} \pmod{64}$ via modular Gaussian elimination. Verification: $P' \cdot (P')^{-1} \equiv I_{10} \pmod{64}$. ✓

Step 6: Key recovery.

$$K = C' \cdot (P')^{-1} \pmod{64}.$$

Recovered key matrix $K \in (\mathbb{Z}_{64})^{10 \times 10}$:

$$K = \begin{pmatrix} 2 & 52 & 52 & 30 & 23 & 49 & 7 & 59 & 9 & 27 \\ 22 & 21 & 8 & 29 & 55 & 5 & 36 & 60 & 13 & 29 \\ 44 & 11 & 6 & 54 & 21 & 25 & 42 & 6 & 5 & 34 \\ 22 & 36 & 28 & 19 & 8 & 40 & 5 & 60 & 47 & 24 \\ 7 & 57 & 55 & 46 & 9 & 26 & 18 & 56 & 6 & 10 \\ 48 & 54 & 49 & 60 & 59 & 33 & 7 & 37 & 34 & 39 \\ 20 & 5 & 7 & 22 & 38 & 21 & 56 & 24 & 4 & 24 \\ 30 & 29 & 3 & 14 & 24 & 18 & 13 & 19 & 31 & 23 \\ 36 & 13 & 57 & 58 & 27 & 42 & 45 & 29 & 50 & 3 \\ 62 & 27 & 58 & 31 & 15 & 51 & 63 & 39 & 1 & 30 \end{pmatrix} \pmod{64}$$

Verification: Applying K to all 20 plaintext vectors reproduces the corresponding ciphertext vectors modulo 64 for every pair. ✓

The SageMath code used:

Listing 2: Known-plaintext attack on Hill cipher over $\mathbb{Z}_{64}$

```

1 # After: load('blockIII_Hill_KPAPairs.sage')
2 # KPAPairs is a list of 20 (plaintext_col, ciphertext_col
  ↪ ) matrix pairs
3
4 R = Zmod(64)
5
6 # Step 2: Build 10x20 plaintext and ciphertext matrices
7 P = matrix(R, [[KPAPairs[j][0][i,0] for j in range(20)]
  ↪ for i in range(10)])
8 C = matrix(R, [[KPAPairs[j][1][i,0] for j in range(20)]
  ↪ for i in range(10)])
9
10 # Step 3 & 4: Select 10 linearly independent columns (
   ↪ found via echelon form)
11 indep = [0, 1, 3, 7, 9, 12, 13, 16, 18, 19]
12 P_prime = P.matrix_from_columns(indep)
13 C_prime = C.matrix_from_columns(indep)
14
15 # Step 5: Compute inverse of P'
16 P_prime_inv = P_prime^(-1)
17
18 # Step 6: Recover the key
19 K = C_prime * P_prime_inv
20 print("Recovered key K =")
21 print(K)
22
23 # Verification: K*P should equal C (mod 64)
24 assert K * P == C
25 print("All 20 KPA pairs verified successfully!")

```

PS-B3-E1: Matrix-based Cryptosystems: Hill Cipher-Q4: Chosen-plaintext attack

Consider a chosen-plaintext attack on the Hill cipher. For a positive natural number n , show that for given key matrix dimension $n \times n$, you can obtain the key with n queries. Argue further why n is an exact number, in the sense that fewer queries are insufficient and strictly more queries not necessary.

1 pts

Answer: Construction with n queries. Choose the n standard basis vectors as the n plaintexts:

$$\mathbf{e}_1 = (1, 0, \dots, 0)^\top, \quad \mathbf{e}_2 = (0, 1, 0, \dots, 0)^\top, \quad \dots, \quad \mathbf{e}_n = (0, \dots, 0, 1)^\top.$$

Querying the encryption oracle on each $\mathbf{e}_i$ yields the ciphertext

$$\mathbf{c}_i = K \mathbf{e}_i = K_{\bullet, i},$$

i.e. exactly the i -th column of K . Hence n queries reveal all n columns and K is fully recovered. More formally, stacking the responses gives $C = K \cdot I_n$, so $K = C \cdot I_n^{-1} = C$.

Why fewer than n queries are insufficient. With only $k < n$ queries we learn at most k columns of K . The remaining $n - k$ columns are completely unconstrained by the observed query–response pairs: for any choice of values for those columns the encryption of the k chosen plaintexts is unchanged. Hence the key cannot be uniquely determined from fewer than n queries.

Why more than n queries are unnecessary. Once we have queried all n basis vectors we have recovered all n^2 entries of K . Any additional query provides only information that is already deducible from K , i.e. it is redundant. Therefore strictly more than n queries are not needed.

Conclusion. The exact query complexity of the chosen-plaintext attack against an $n \times n$ Hill cipher is **n** : sufficient to recover K completely, and necessary since any $k < n$ queries leave at least one column undetermined.

PS-B3-E2: Vigenère Cipher

A poly-alphabetic substitution cipher uses multiple simple substitutions to encrypt a message. The Vigenère cipher is a classic poly-alphabetic substitution cipher, where its main purpose is to distribute the frequencies so that the characters in the ciphertext are all likely to occur around the same number of times. For more details in the mechanics and the underlying structure of the Vigenère cipher, please see: <https://caesarcipher.org/ciphers/vigenere>

PS-B3-E2: Vigenère Cipher-Q1: Attack using Frequency Analysis

Suppose that the following ciphertext was created using a Vigenère cipher (blank spaces are not encrypted):

```
GZEAKAVKWNLCUANTQOHJITZTAGWOMTQGNGOOEJYGZELSRRLQ
BTUFQDRUICZEEANTQOHJGEGUCHRRKEALAGAOAKLVVEFLOAGT
VUEGZIFEEFKATWPEGFRKSBJSZATUKALKTUSTIAGRFEWIFMN
OJENCAODEOMTUWFBJGBLTBLEYDYBMAOGUGXRRRIURFCLSNNDY
FASNFDXSSVKKVLEFLSNJEPGMVFGNFDLGUEYRBMPFLIYDHNKN
GXIAASUWDGZEOWAZWRFDIQWSGZEFWCEWTCSSFOOEVTBLHRMN
VNEEKIGQWVXIVKPNKSJGRQGNRLWBLHEWESGUEXIIWSVPSRNE
AWITZTAANRREEGBHLSUZHQNGNGLEYDAAQOAWEFHEPAAFYAGT
GZEVLDRAELMRFTVXYBMAEWRSDVFGGZIFQHSRRHRBTAODY
NURLHTBYRNHHLKTHVEALOESENAKANYEALTEQIAYTBJEPJUVLY
BMRTJOHHFBJAGGPFWEWTZASFAOANIGLIANTHIMRANFDSJE
RUOSXERJEZWMOWRGZAGLHRTEFLWNQTBZIQWAZWSFSGRASGGP
HLIGANLGUEHRRKEALAGAOATEPSUFWNBTOQRRSDFLHRXIAWP
EANGSNLOALLHRUINGNPWLBKTNEEFKATWIASTJWEGSBBMTCAZ
MSTUWVVYEAWRRUICZEEASYAKRSCNWSNJSNDAQTUGOIGZMBJE
YWTGMCRSNQDEFKDEWSFANTSNSQLFGIGMSRKAXWYJGRQLHNL
BMRTJOHHPEGBNTLLXOEYOGLONYRRWOATYGZEJSYGZEUADQWN
ZWSFSGRANGZIFLEKLIFSTGZEIWRNLWNQTUGQOHZAIWTBVEPAP
UWRVLFVJSGLOSANQGUGOHNLYBMRAWXGSSFAGAEALWVDLOWT
UWNFSHNKAOSCXVOBJIAQOHJLNHTBHBHLIGGNYQAPLIISTRKW
UWNLGUBHEAHOJWRCGIALSBTEPSRRXUYOIGZBYMRCEJFWNGST
VGNFCIYDSBJTUWYJALYCNBOCEQPGGGESPUQIFWAFQUALIYQO
HJENDIMWTUSTRNEEQGEGUCANPDAFKIFMSVFGGZEFSMRCELOO
EVWUACUASCJOOSBYQTUWNNEEBXTUWCBXFRWSUGPAWXGVOBJH
VVDRFMRKSNYEFLEALYBMAWXGSSFAGAEALIFLOPJAPCTUAS
PAPUWRNFDCJEFWNGZOJQOHVIQATVFFVNEZANHLEFGRYWSFFE
KLTVEEUADQWNZWSFSGRWNQ.
```

Perform a cryptanalysis of this ciphertext and consider the following:

1. What keyword length has been used? (2 points)

2. Use <https://legacy.cryptool.org/en/cto/vigenerebreak> functionality of CrypTool-Online to decrypt the ciphertext provided above. (4 points)

6 pts

Answer: YOUR-SOLUTION-HERE

PS-B3-E3: DES

The *data encryption standard* has been used widely in the past, but is considered a legacy cryptosystem today. In particular, it is NOT (!) considered *secure* anymore! However, both its design as well as its modes of operation still provide valuable insights into cryptographic design even today from an educational perspective. For this exercise, please make use of the Avalanche (DES) template in CrypTool 2 to assist you.

PS-B3-E3: DES-Q1: Mechanics of bit manipulation in DES

- What is the output of the first round of the DES algorithm when the plaintext and the key are both all zeros?
- What is the output of the first round of the DES algorithm when the plaintext and the key are both all ones?

4 pts

Answer: YOUR-SOLUTION-HERE**PS-B3-E3: DES-Q2: Avalanche effect in DES**

Remember that it is desirable for good block ciphers that a change in one input bit affects many output bits, a property that is called diffusion or the avalanche effect. We try now to get a feeling for the avalanche property of DES. We apply an input word that has a “1” at bit position 57 and all other bits as well as the key are zero. (Note that the input word has to run through the initial permutation.)

1. What is the output after the first round?
2. How many output bit after the first round have actually changed compared to the case when the plaintext is all zero? (Observe that we only consider a single round here. There will be more and more output differences after every new round. Hence the term avalanche effect.)

4 pts

Answer: YOUR-SOLUTION-HERE

Appendix: Instructions for Homework Sheet Preparation in L^AT_EX

0.1 How to Include an Image

A picture can be included in your document using the L^AT_EXcode used to obtain Figure 1.

0.2 How to Include a Table

A table can be included using the commands used to obtain Table 1.

Software	Assessment	Comments
Suite A	works well	internal use only
Suite B	freely usable	observe guidance document
Suite C	restricted	Requires compiler

Table 1: Caption

0.3 How to List Data like Hexdumps

Hexadecimal encoding provides a way to encode data in a machine and human-readable way. The code that has been used to generate Listing 3 can be used to include hex-data, including the one provided by CrypTool1.

Listing 3: Example for how to include hex-data.

```

1 50 63 D8 D0 D8 E4 FC F7 09 FB 1F 40 C1 99 E7 62 56 8A 20 77 B9 D6 F7 21 B3 61 F0 8D
2 96 AA 45 E3 A0 A5 11 97 58 17 2B 53 60 6D A3 2C A0 4C 2E 1F 5F 31 94 6A CC 09 28 8E
3 E0 0E 7C 23 2E 2F E3 E6 6A 02 AA D2 A7 41 80 6D AF 5D 72 63 0E AE A8 1F 3A 38 E3 91
4 74 56 B6 D7 59 C8 14 4B 0F C2 D8 9F 46 98 26 92 AF 5D 60 9E 5D AC 30 A1 DD 7A D1 95
5 49 AD 54 3C 93 B0 DF BC B2 57 83 AF F7 43 D8 98 0B 06 A4 6B 42 4C 8D 44 F4 7C BB B0
6 A6 B3 A2 7E 9C 11 D7 A6 98 23 EC 23 8E C8 AC FD 15 B0 2F 2B EC 49 25 B5 70 9C 0B D5
7 8A 24 5F 8C 7D 1A 32 E9 4C EF E7 B1 06 C7 D4 A2 C8 DA 01 15 99 E2 92 6B 0E DF 5B 36
8 15 2B 30 30 E0 5A A6 E0 80 2F 07 4F B4 4F 66 F8 17 A0 A3 72

```

figures/cryptool1.PNG

Figure 1: The code of this figure can be used when preparing your homework.